Ders 11 Genişletilmiş Notu: Kelimelerden Cümle Üretimi Sunumları, Memory Networks, QANet, Sohbet Robotları ve Sanal Asistanlar

Ders 11 Genişletilmiş Notu: Kelimelerden Cümle Üretimi Sunumları, Memory Networks, QANet, Sohbet Robotları ve Sanal Asistanlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ödev Sunumları: Üç Kelimeden Cümle Üretimi
- 2. Memory Networks
- 3. End-to-End Memory Networks
- 4. Arka Kapılar Üzerine Tekrar
- 5. QANet ve SQuAD
- 6. Bu Çalışmalar Neden Önemli?
- 7. Sohbet ve Sanal Asistan Uygulamaları
- 8. Diyalog Derlemleriyle Cevap Bulma ve Soft Cosine
- 9. Diyalog Üretimi ve Microsoft Tay
- 10. Görev Tabanlı Sanal Asistanlar
- Örnek Sorular
- Dersin Toparlaması

1. Ödev Sunumları: Üç Kelimeden Cümle Üretimi

Dersin ilk yarısı ödev sunumlarına ayrıldı. Ödevde üç kelime verilip bu üç kelimeyi içeren bir cümle üretmek isteniyordu. Sunumlarda Markov zinciri, LSTM, fastText ve T5 denendi. Hoca her sunumda yöntemin ayrıntısını sorularla netleştirdi; bu soru-cevaplar aşağıda adsız verilmiştir.

1.1 Markov zinciri ile 1-gram ve 2-gram karşılaştırması

Bir grup, hocanın paylaştığı roman derlemlerinden birini kullandı. Yöntem olarak Markov zincirinin iki hâlini karşılaştırdı: 1-gram (bir sonraki kelimeyi yalnız son kelimeye bakarak seçen) ve 2-gram (son iki kelimeye bakarak seçen). Bağlaçlar ve benzeri kelimeler temizlenmediği için “ve”, “de” gibi kelimeler de kelime olarak alındı.

Algoritma adımları şöyleydi:

1. Derlemden rastgele üç kelime seç.
2. Zinciri bu kelimelerden biriyle (genelde ilkiyle) başlat.
3. 1-gramda son kelimeye, 2-gramda son iki kelimeye bakarak sonraki kelimeyi tahmin et; en çok 200 kelimeye kadar üret.
4. Üretim sırasında diğer iki kelimenin metinde geçip geçmediğini kontrol et. Üçü de bulunursa cümle tamamlanana kadar devam et ve kelime üçlüsünü ve cümleyi kaydet. Bulunamazsa yeni bir kelime üçlüsü seç.

Soru: Sisteme üç kelime veriyorsunuz, zinciri birinci kelimeyle başlatıyorsunuz. İkinci ve üçüncü kelimenin cümlede olması gerektiğini sisteme nasıl söylüyorsunuz? **Cevap:** Söylenmiyor. Sistem bir kelimeyle başlayıp üretiyor, sonra üretilen metinde diğer iki kelimenin olup olmadığı kontrol ediliyor. Kelimeler arasında bir sıra da verilmiyor. Hoca bunu "aslında bağlamaya çalışmıyor, ürettiğinin içinde onlar var mı yok mu diye kontrol ediyorsunuz" diye özetledi.

Sonuçlar: 2-gram belirgin biçimde daha anlamlı cümleler üretti. 2-gram 50 kelime üçlüsünün hepsinde cümle üretebildi, 1-gram ise bazı üçlülerde 200 kelime sınırı içinde üç kelimeyi bir araya getiremedi. Grup, derlemde bulunmayan ve algoritmanın kendisinin kurduğu bir cümleyi özellikle gösterdi.

Soru: Üretilen cümleler derlemde geçiyor mu, baktınız mı? **Cevap:** Bazı cümleler romandaki bir kalıbı başta ya da sonda aynen alabiliyor; gösterilen cümle ise derlemde yoktu.

Soru: "Oluşturamadı" ne demek? **Cevap:** 200 kelime sınırı içinde o üçlüyle cümle kurulamadı. Sınır büyütülürse ya da tekrar denenirse bulunabilir ama çok uzun sürer. Karşılaştırma eşit olsun diye her üçlü tek seferde denendi.

Grubun sonuç slaytı (ekrandan, 00:10:00):

- Üç roman derlemi birleştirildiğinde 200 kelimelik cümlelerin üretim süresi ciddi biçimde arttı, bu yüzden küçük bir derlem kullanıldı. Tek derlemde toplam token 56.000, farklı token 15.000; üç derlemde toplam token 435.000, farklı token 78.000.
- 20 kelime sınırıyla cümle üretmek çok uzun sürdü ve anlamsız cümleler çıktı. Görece anlamlı bir örnek: "Bir düşün, korkunç ölüm laneti var, bugüne kadar çocukları korkutmaktan başka bir ismi taşıyan iki yabancı olarak yaşıyoruz aynı evde yaşadı."
- 2-gram ile 1-gram karşılaştırıldığında 2-gramda daha anlamlı örnekler görüldü. Özgün cümle "Körle kötürümün yoldaşlığı gibi bir iş için değil!" iken 2-gramın ürettiği cümle "Körle kötürümün yoldaşlığı gibi bir gerçek vardı".
- Kelime üçlüleri rastgele seçildiği için cümlelerin çoğunda anlam bozuklukları var; üçlüleri elle belirlemek sonuçları iyileştirebilir.

Soru: 20 kelime sınırı ile 200 kelime sınırı arasındaki fark ne, neden 20'de çok uzun sürüyor? **Cevap:** Üç kelime rastgele seçiliyor. 200 kelimelik metinde üçünün birden geçme olasılığı daha yüksek. 20 kelimede çoğu zaman üçü bir araya gelmiyor, sistem yeni bir üçlüye geçip yeniden deniyor. Hoca bunu "böyle bir cümleyi bulma olasılığı düşüyor, dolayısıyla daha fazla deneme yapmanız gerekiyor" diye toparladı.

Soru: Program deterministik mi? Aynı kelimeyi verip defalarca çalıştırınca farklı sonuçlar alıyor musunuz, bunu ne sağlıyor? **Cevap:** Farklı sonuçlar alınıyor. Geçiş matrisi derlemden bir kez oluşturuluyor. Her adımda, mevcut kelimeden sonra gelebilecek kelimeler arasından olasılıklarına göre ağırlıklı rastgele bir seçim (weighted choice) yapılıyor. Yani en olası kelime her zaman seçilmiyor.

1.2 Markov zinciri, LSTM ve T5

Bir öğrenci Markov zinciri, LSTM ve T5'i denedi. Markov zincirinde öncekine benzer bir yol izledi: üç kelimenin birlikte geçtiği bir metni üretmeye çalıştı. Üçünün birlikte bulunma olasılığı çok düşük olduğu için üretilen metinler çok uzun oldu. N büyüdükçe daha kısa ve anlamlı örnekler görüldü.

LSTM'de girdi, derlemdeki 25 kelimelik pencereler; çıktı, ardından gelen kelimeydi (next word prediction). Üretimde bir kelimeyle başlanıp birkaç kelime üretiliyor, sonra verilen ikinci kelime araya zorla ekleniyor ve üretim oradan sürüyor. Öğrenci, Markov'un mevcut derlemdeki geçişleri izlediği için derleme çok benzeyen cümleler ürettiğini belirtti.

T5 tarafında İngilizce hazır bir modelle üç kelimeden çok anlamlı cümleler üretilebildi, ama Türkçe için eğitim verisi kurulamadı. Hoca çok dilli bir T5 (multilingual T5) bulunduğunu hatırlattı ve şu öneriyi yaptı:

Soru (hocadan öneri): Türkçe eğitim verisi şöyle oluşturulamaz mı? Rastgele bir cümle al, içinden rastgele üç kelime seç, bu üç kelime girdi dizisi (input sequence) olsun; çıktı dizisi (output sequence) ise cümlenin kendisi olsun. **Cevap:** T5'in bütün görevleri sekanstan sekansa

(sequence to sequence) olduğu için bu yapı uygundur. Girdi üç kelime, çıktı o üç kelimenin birlikte geçtiği cümledir. Öğrenci bu açıdan bakmadığını, doğru olduğunu kabul etti.

Hoca sunumda yapılanların slaytlarda net görünmediğini de belirtti.

1.3 Kelime tabanlı Markov zinciri ve karakter tabanlı LSTM

Bir öğrenci hocanın paylaştığı altyazılar veri kümesiyle karakter tabanlı LSTM ile kelime tabanlı Markov zincirini karşılaştırdı. Üç kelimeden biri cümlenin başına konuyor, Markov zinciri en fazla üç sözcük üretiyor, sonra ikinci sözcük ekleniyor ve böylece üç sözcük de kullanılıyordu. Karakter tabanlı LSTM'de de kelimeler araya elle eklenip o karakter dizisi verilerek üretim sürdürüldü.

Soru: Markov'un varsayımı kaç, yani kaç önceki kelimeye bakıyor? **Cevap:** Bir önceki sözcüğe bakıyor (bir önceki gruptaki 1-gram gibi).

Sonuç: karakter LSTM'de ekler yanlış olabiliyor, hatta var olmayan sözcükler çıkabiliyordu. Markov zincirinde cümleler bütün olarak anlamlı değildi ama ikili, üçlü, dörtlü kelime grupları anlamlıydı. Öğrenciye göre bu denemede Markov daha başarılı oldu. Eğitim uzun sürdüğü için az veri kullanıldı.

Hoca karşılaştırmanın tam adil olmadığını belirtti. İki şey birden değişiyor (karakter/kelime tabanı ve yöntem). Karakter LSTM karakterlerden kelime üretmeyi tam öğrenememiş; derlem ya da eğitim süresi yetmemiş görünüyor. LSTM de kelime tabanlı kullanılsaydı fark daha net görülürdü.

1.4 Film yorumlarıyla Markov zinciri ve LSTM

Bir grup film yorumları derleminden yararlandı. Slayta göre derlemden 1500 yorum kullanıldı; tüm derlemde 57 bin pozitif ve 25 bin negatif yorum var (ekrandan, 00:30:00). Bellek ve süre sınırları yüzünden yalnız 1500 yorum kullanılabildi, pozitif/negatif ayrımına bakılmadı.

Ön işleme adımları: küçük harfe çevirme, link ve hashtag'lerin kaldırılması, noktalama işaretleri ve özel karakterlerin kaldırılması, fazla boşlukların silinmesi, Türkçe karakterlerin İngilizce karakterlere dönüştürülmesi, satır sonu karakterlerinin kaldırılması.

Markov zinciri için cümleler token'lara ayrıldı. Bir DataFrame'de "word", "word+1" ve "freq" sütunları oluşturuldu, yani ardışık kelime ikilileri ve sıklıkları tutuldu. Sonra satırları "word", sütunları "word+1" olan bir pivot tablo kuruldu ve olasılıklar bu tabloda saklandı. Grubun verdiği örnek: "Ali top oynadı" cümlesinde "Ali top" ikilisi 6 kez, "top oynadı" ikilisi 4 kez geçiyorsa olasılıklar 0,6 ve 0,4 olur.

> **Ders sonrası not:** Grup bu örnekte farklı önceki kelimelere ait iki ikiliyi birlikte normalize etti. Markov zincirinde geçiş olasılığı aynı önceki kelimeye göre normalize edilir: $P(\text{oynadı} \mid \text{top}) = c(\text{top oynadı})/c(\text{top})$. Pivot tabloda bu, her satırın kendi içinde normalize edilmesi demektir.

Üretim algoritması: kullanıcıdan üç kelime al, ilk kelimeden başla, pivot tablodan sonraki kelimeyi olasılığına göre seç. Üç kelimeyi aynı cümlede üretmek için cümleler çok uzuyordu, bazen hiç bitmiyordu. Bu yüzden cümle uzunluğu 15 kelimeyle sınırlandı. Bir de kural eklendi: uzunluk 10'un katıysa son kelimeyi, 5'in katıysa ortadaki kelimeyi cümleye ekle. Bir cümle sonu fiilleri listesi tutuldu; kelime bu listedeyse ve cümle yeterince uzunsa üretim bitti.

LSTM için cümleler 10 kelimelik dizilere bölündü: 0-9. indisteki kelimeler bir girdi, 10. indisteki kelime hedef; sonra bir kaydırılarak 1-10 girdi, 11 hedef ve böyle devam. Optimizer olarak Adam, aktivasyon olarak softmax, kayıp fonksiyonu olarak categorical cross entropy kullanıldı. Cümle üretiminde Markov ile aynı mekanizma uygulandı.

Örnekler slayttaki tabloda (ekrandan, 00:40:00):

Kelimeler	Markov Chain	LSTM
güzel, robot, son	güzel olursa olsun ama işte robot taklidi yapanların kavgasıydı bende son zamanlarda izledim	güzel de yinede dunyasinin istiyorsaniz robot sasirtici ve ayrica kocaman son puan
film, mutlu, çok	film . bilgisayar teknolojisinin nimetlerinden mutlu sonla bitiyor gerçekten hayranlık çok iyiydi kesinlikle …	film eglenceli sinemaya olusturulsaydi ana mutlu cok acikcasi sahnesinde oldugu mutlu devam
film, güzel, iyi	film tavsiye etmem . filmlerini güzel işte mafya karakterini oynayan adam iyi olabilirmiş. mikail iyi helal olsun	film guzel yaninda isleyen sahneler sozleri yasanir cok dava tebrik iyi sevdigim
ben, güzel, iyi	ben begendim . seyircisine sert güzel bir filmdi diyecek kadar iyi . cumartesi gunu iple	ben birlikte izlemelisiniz filmi ve guzel harika bence kendi alip iyi bakalim

Grubun çıkarımı: Markov zinciri LSTM'den daha iyi cümleler üretti. Markov'da model eğitimi yoktur ve ikili olasılıklar dili bu küçük veride daha iyi modelledi. LSTM bir öğrenme modeli olduğu ve çok az veriyle eğitildiği için iyi sonuç vermedi; daha büyük veri ve daha uzun eğitimle daha iyi olabilirdi.

Soru (bir öğrenciden): LSTM'de "güzel", "robot" ve "son" için ayrı ayrı cümleler üretip birleştirdiniz mi? **Cevap:** Hayır, cümle değil kelime tahmini yapılıyor. "Güzel"den sonra tahmin yapılıyor; belli bir noktada "robot" gelmemişse dizi kesilip "robot" ekleniyor. Bundan sonra next word "robot" oluyor ve tahmin ondan sonra sürüyor. Cümleler üretilip birleştirilmiyor. Markov tarafında da aynı mekanizma kullanıldı.

1.5 Mesnevi üzerinde word2vec ve fastText embedding'li LSTM

Bir öğrenci kelime tabanlı LSTM kullandı ve iki modeli embedding katmanına göre ayırdı: birinde word2vec ile, ötekinde fastText ile üretilmiş ön eğitimli ağırlıklar kullanıldı. Derlem olarak Mevlana'nın Mesnevi'si seçildi. Seçim gerekçesi hocanın önceki derste söylediği bir gözlemdi: düz metinden üretilen cümlelerin yapay olduğu kolayca anlaşılır, ama şiir benzeri bir metinde okur "derin bir anlam bırakılmış" gibi algılayabilir. Mesnevi de satır satır şiir gibi ilerler.

Eğitim verisi kelime bazlı hazırlandı. Örneğin "göçme zamanı geldi" cümlesinde "göçme zamanı" girdi, "geldi" etikettir.

Üretim yöntemi diğerlerinden farklıydı:

1. Derlemden rastgele üç kelime seç (örnekte "hakim", "hayır", "kötülükle").
2. Bunlardan derlemde en sık geçenle (örnekte "hayır") modeli kullanarak 10 kelimelik bir "çerçeve" cümle üret. En sık kelime seçilir, çünkü onunla başlayan cümlelerde diğer kelimelerin uygun bir yer bulma olasılığı daha yüksektir.
3. Diğer iki kelimeyi yerleştirmek için çerçevedeki her konumu dene. Örneğin "hayır"dan sonra "hakim" gelme olasılığı ne, "dedi ki"den sonra ne? Model, derlemdeki bütün kelimeler için bir tahmin değeri verir. Kelime, olasılığın en yüksek olduğu konuma yerleştirilir.

Bu yöntemde arama yoktur: bir cümle üretilir, sonra kelimeler en uygun yerlere konur. Öğrencinin örnek çıktısı: "Hayır, dedi ki hakim yemek tasavvuf zatından doğan biten kurdun sevdiklere kötülükle kışı."

Rastgele seçilen kelimeler birbiriyle ilgisiz olabildiği için anlamlı cümle çıkmayabiliyordu. Öğrenci kendisi birbirine yakın kelimeler verdiğinde (örneğin "peygamber", "dost", "lütfet") daha anlamlı sonuçlar aldı. Gözlemleri:

- word2vec, fastText'e göre daha anlamlı cümleler üretti.

- fastText'te farklı üçlülerde bile aynı kalıpların tekrarlandığı görüldü.
- Cümlenin verilen kelimelerden biriyle başlamak zorunda olması bir kısıttı, ama diğer çalışmalarda da durum böyleydi.
- Çerçeve 10 yerine 20 kelime olsaydı kelimeler daha anlamlı yerlere yerleşebilirdi.

Hoca word2vec'in en azından bu görev için fastText'ten daha iyi olduğunun gösterilmesini güzel bir yorum buldu. Ama yöntemin iyi çalışması için çerçeve cümlenin kendisi çok iyi olmalıdır, çünkü araya kelime sokmak cümleyi bir şekilde bozar. Gösterilen çerçeve cümle tek başına anlamlı değildi. Hocaya göre derlem büyüklüğünün etkisi büyük; girdi penceresi ve derlem daha büyük olsaydı muhtemelen daha iyi cümleler üretilirdi.

1.6 fastText vektör ortalamasıyla metin büyütme ve LSTM

Son sunumda bir öğrenci iki yöntem kullandı. Diğerlerinden farklı olarak üç kelimeyi araya serpiştirmedi; üç kelimeyi başlangıç tohumu (seed) kabul edip metni onlardan devam ettirdi. Derlem bir yarışma eğitim verisiydi; tweet'lerden oluşuyordu.

- **fastText yöntemi:** Derlemden bir fastText modeli eğitildi. Tohumdaki kelimelerin vektörlerinin ortalaması alındı, bu vektöre en yakın kelime bulunup tohuma eklendi ve böylece metin büyütüldü. Markov ya da sıralı bir model kullanılmadı.
- **LSTM yöntemi:** N önceki kelimeden sonraki kelimeyi tahmin eden standart yapı.

Sonuçlar: LSTM sıraya dayalı olduğu için anlamca ilişkili tohumlarda daha akıcı cümleler kurdu (bir örnek: "çok yetersiz geldi … hizmetleri …"). Birbiriyle ilgisiz kelimeler verildiğinde ("hayvan", "çocuk", "hava" gibi) fastText yaklaşımı bir tık daha iyi göründü, çünkü orada zaten bir anlam ilişkisi yok ve yöntem yakın kelimeleri ekleyerek ilerliyor. Hoca genel beklentinin LSTM'in daha başarılı olması olduğunu, çünkü ardışıklığı modellediğini söyledi. Üretilen metinler yeni tweet'ler olarak düşünülebilir.

2. Memory Networks

2.1 Hatırlatma: bAbI veri kümesi

Önceki derste soru cevaplama sistemlerinin çıkarım yapıp yapamadığını ölçmek için Facebook'un araştırma laboratuvarında hazırlanan bAbI veri kümesinden söz edilmişti. Bir hikaye (birkaç cümle) ve bir soru verilir; sistem cevabın hangi kelime olduğunu bulmalıdır.

Slayttaki bilgiler (ekrandan, 01:10:00): otomatik üretilmiş (eğitim 3000, test 1000 soru), sözlük boyutu çok küçük, çok basit bir gramer. Eğitim verisine bir örnek (çeviri):

```
1 Mary banyoya gitti.
2 Sandra bahçeye gitti.
3 Daniel bahçeye gitti.
4 Daniel ofise gitti.
5 Sandra oradaki sütü aldı.
6 Sandra sütü oraya bıraktı.
7 Süt nerede?      bahçe      5 6 2
```

Son sütun cevapla ilgili cümlelerin indisleridir. Sözlükte en fazla 100 civarında farklı kelime vardır. Örnekler kurallara göre kodla otomatik üretilir; bir yerden toplanmamıştır. Eğitim verisinde girdi olarak cevaba ulaşmak için hangi cümlelerin kullanılması gerektiği de verilir.

> **Ders sonrası not:** 01:10:00'daki karede indisler "5 6 2" olarak görünüyor. Hoca iki adımlı yapıyı anlatırken örnek olarak 5 ve 6'yı kullandı ve 01:40:00'daki karede slaytta "5 6" görünüyor. "Oraya" kelimesinin bahçe olduğunu bilmek için 2. cümle de gereklidir.

Önceki derste bu veri kümesinde memory networks'ün kullanıldığı söylenmiş ama iç yapısı anlatılmamıştı. Bu derste ona dönüldü.

2.2 Bileşenler

Memory Networks 2014'te yayımlandı (arXiv:1410.3916). Slayttaki tanımlar (ekrandan, 01:20:00):

- YSA + memory mimarisi; hafızaya yalnız yazılır ve okunur, değiştirme mekanizması yoktur.
- m: hikayenin tutulduğu hafıza. m_i hikayenin her bir satırıdır.
- m_{o1} ve m_{o2}: hafızanın cevabı üretecek parçaları. Örnekte 5 ve 6 numaralı cümleler gibi.
- x: soru.
- r: tek kelimelik cevap.
- s_O ve s_R fonksiyonlarının U matrisleri farklıdır.
- U matrisleri eğitim verisinden öğrenilir; U, $n \times d$'lik bir matristir.
- Θ (teta) fonksiyonu bir string'i $d \times 1$ boyutlu gösterime çevirir; kelime torbası (BoW). Sözlük çok küçük olduğu için d'nin 100 civarında olduğu düşünülebilir; bu gösterim one-hot kodlama gibidir.

2.3 Benzerlik fonksiyonu $s(x, y)$

$$s(x, y) = \Phi_x(x)^\top U^\top U \Phi_y(y)$$

Hoca çıktının ne olduğunu boyutlardan kurdu:

- $\Phi_x(x)$ $d \times 1$ boyutludur; transpozu $1 \times d$ olur.
- U $n \times d$ olduğu için $U^\top$ $d \times n$ boyutludur.
- Ardından U gelir: $n \times d$.
- En sonda $\Phi_y(y)$: $d \times 1$ (transpoz yok).

$$(1 \times d)(d \times n)(n \times d)(d \times 1) = 1 \times 1$$

Yani s tek bir sayı üretir. Bu sayı, verilen x string'i ile y string'inin birbirine benzerliğini ölçer. Aradaki U öğrenilir.

U'nun rolü şudur: U olmasa ve iki one-hot vektörün yalnız iç çarpımı (dot product) alınsa, benzerlik yalnız ortak kelimelerden gelirdi. U ile çarpmak, word2vec'te bir kelimenin one-hot gösterimini bir W matrisiyle çarparak temsilini bulmaya benzer. Bu sayede benzerliğe anlamsal bir boyut katılır. Burada her iki taraf da çarpılır, sonra birbiriyle çarpılarak benzerlik hesaplanır.

2.4 argmax ve çıkarım adımları

Slayttaki formüller (ekrandan, 01:20:00):

$$o_1 = O_1(x, \mathbf{m}) = \arg \max_{i=1,\ldots,N} s_O(x, \mathbf{m}_i)$$

$$o_2 = O_2(x, \mathbf{m}) = \arg \max_{i=1,\ldots,N} s_O([x, \mathbf{m}_{o_1}], \mathbf{m}_i)$$

$$r = \arg \max_{w \in W} s_R([x, \mathbf{m}_{o_1}, \mathbf{m}_{o_2}], w)$$

Soru (hocadan): argmax ne demek? **Cevap:** argmax, yanındaki fonksiyonun ($s_O(x, m_i)$) değerini en büyük yapan i değerini seçer. i, 1'den N'ye kadar değer alır; N hafızadaki cümle sayısıdır. argmin ise fonksiyonu en küçük yapan değeri seçer.

- o_1**:** Soruya (x) en çok benzeyen hafıza satırıdır. "Süt nerede?" sorusu için yedi satır içinden en benzeyeni getirir.
- o_2**:** Bu kez soru ve m_{o1} birlikte tek bir string olarak alınır (örneğin "Süt nerede?" ve "Sandra oradaki sütü aldı."). Bu ikisine birden en çok benzeyen satır bulunur. Sistem iyi çalışırsa o_1 ve o_2 5 ve 6 numaralı cümleler olmalıdır.
- r**:** Bu kez seçim büyük W'dan, yani sözlükten yapılır. Sözlükteki her kelime denenir ve x, m_{o1}, m_{o2}'nin üçüne birden en çok benzeyen kelime cevap olarak seçilir.

Bu yapı, cevaba iki cümle kullanılarak ulaşılan görev için kurulmuştur. Daha fazla cümle gerekirse o_3, o_4, o_5 diye ilerler.

2.5 Eğitim ve kayıp fonksiyonu

Yukarıdaki adımlar eğitim bittikten sonra, test zamanında yapılan işlemlerdir: yeni bir x geldiğinde önce o_1, sonra o_2, sonra r bulunur. Eğitimde bir kayıp fonksiyonu tanımlanmalı ve ağırlıklar (en azından U matrisleri) ona göre güncellenmelidir.

Slayttaki kayıp fonksiyonu üç satırlıdır; bu sayı da cevaba ulaşmak için gereken cümle sayısının iki olduğu varsayımından gelir. İlk satır x ile gerçek m_{o1} arasındaki benzerliği, gerçek olmayan bütün seçeneklerle ($\bar{f}$) karşılaştırır. Diğer satırlar aynı şeyi m_{o2} için ve cevap için ($\bar{r}$: gerçek cevap dışındaki bütün kelimeler) yapar. Formüldeki ek bir terim değerlerin sıfıra gitmemesi için eklenmiştir. Bu kayıp, U'ları bulmak için tasarlanmıştır.

> **Ek bilgi (derste anlatılmadı):** Memory Networks makalesinde bu kayıp büyük olasılıkla marjinli sıralama (margin ranking) kaybıdır: her terim $\max(0, \gamma - s(\text{doğru}) + s(\text{yanlış}))$ biçimindedir. Doğru seçeneğin skoru, her yanlış seçeneğin skorundan en az γ kadar büyük olana dek ceza verilir.

Burada önemli bir kısıt vardır: eğitimde cevabın bulunabilmesi için gerekli cümlelerin indisleri (5 ve 6) sisteme verilmelidir, çünkü kayıp onlar kullanılarak hesaplanır. Testte bunlara gerek yoktur; sistem onları kendisi bulur.

3. End-to-End Memory Networks

3.1 Temel fikir

Bir yıl sonra End-to-End Memory Networks ortaya atıldı (arXiv:1503.08895). Slayttaki ana iddia (ekrandan, 01:25:15): eğitim setinde cevapla ilişkili cümlelerin (m_{o1}, m_{o2}) işaretlenmesi gerekliliğini ortadan kaldırır. Benzer bAbI görevlerinde eğitim kümesinde yalnız hikaye, soru ve cevap vardır.

$$p_i = \text{Softmax}(u^\top m_i), \qquad \text{Softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

- m_i: memory slotları (önceki m_i'lerle aynı, hikayenin cümleleri).
- u: sorunun embedding'i.
- p: memory slotlarının soruyla ilişkili olma olasılıkları.

Softmax, toplamı 1 olan olasılıklar üretir. u bütün slotlar için aynıdır; her m_i ile çarpılır. Soruya benzerliği yüksek olan m_i'nin p_i değeri de yüksek olur. Soruya benzeyen birkaç cümle olabilir; onların olasılıkları sıfırdan büyük, diğerlerininki sıfıra yakın olur, ama toplam yine 1'dir.

3.2 Tek adımlı mimari

Slayttaki şekil ve formüller (ekrandan, 01:30:00):

$$o = \sum_i p_i c_i, \qquad \hat{a} = \text{Softmax}(W(o + u))$$

Eğitim sürecinde A, B, C, W embedding matrisleri öğrenilir. Akış şöyledir:

1. Soru q one-hot kodlanır ve B embedding matrisiyle çarpılarak istenen boyuta indirilir; bu u'dur.
2. Hikayedeki her cümle (x_i) one-hot hâlden A matrisiyle çarpılarak d boyutlu m_i'ye dönüşür. Örneğin 50 cümle varsa her biri $1 \times d$ boyutlu bir vektör olur.
3. Bütün m_i'ler u ile iç çarpıma (inner product) sokulur ve softmax uygulanır; bu p_i'leri verir.
4. Aynı cümleler bir de C embedding matrisiyle başka bir uzaya taşınır ve c_i'ler elde edilir. Yani aynı cümleler için iki embedding uzayı oluşturulur.
5. p_i'ler ile c_i'ler çarpılıp toplanır (ağırlıklı toplam); tek bir o vektörü elde edilir.
6. o ile u toplanır, öğrenilen W matrisiyle çarpılır ve softmax uygulanır; bu $\hat{a}$, yani tahmin edilen cevaptır (predicted answer).

p vektörünün eleman sayısı hikayedeki cümle sayısıdır. $\hat{a}$ ise sözlük boyutludur: her kelimenin cevap olma olasılığını verir ve en yüksek olanı cevap olarak seçilir.

3.3 Üç adımlı (3 hop) mimari

Standart yapı tek adımlıdır: soruya en çok benzeyen hafıza bölgesi bulunur ve cevap onun bir temsiliyle soru birlikte kullanılarak tahmin edilir. Ama bAbI'de birden fazla cümle kullanmak gerekebilir. Bu yüzden yapı üç adımlı hâle getirilmiştir. Önce soruya en çok benzeyen bölge bulunur, sonra o kullanılarak ikincisi, onu da kullanarak üçüncüsü bulunur.

Hoca bir inceliğe dikkat çekti: tek adımlı yapı da aslında birden fazla m_i'yi kullanabilir. p tek bir 1 ve geri kalanı 0 olan bir vektör değildir; örneğin bir slotta 0,5, birinde 0,4, diğerlerinde çok küçük değerler olabilir. O zaman ağırlıklı toplam iki slotu birden içine katar. Üç adımlı yapı bu imkânı daha da artırır. Değişen tek şey, A ve C yerine her adım için ayrı $A^1, C^1, A^2, C^2, A^3, C^3$ matrislerinin öğrenilmesidir.

Özetle: memory networks'te eğitimde dışarıdan verilen m_o'ları bu yapı kendisi seçer. Tek adımlı yapı tek bir m_o kullanımına (içinde birden fazla m_i olabilir), üç adımlı yapı üç m_o kullanımına karşılık gelir. Her adımın ayrı A ve C embedding matrisleri vardır.

3.4 Cümle temsili: BoW ve position embedding

Cümlelerin temsilinde iki yol vardır. Birincisi kelime torbasıdır (BoW): cümle one-hot hâlde A matrisiyle çarpılır. Bu, cümlede geçen (sıfır olmayan) kelimelerin A'daki d boyutlu karşılıklarının toplamını almak demektir. d embedding boyutu, V sözlük boyutudur. A yerine word2vec gibi hazır bir matris de kullanılabilir.

Soru (hocadan): x'in içindeki kelimelerin yerlerini değiştirsem elde edeceğim m_i değişir mi? **Cevap:** Değişmez. Vektörler toplanıyor ve toplamada sıranın önemi yoktur: $3+5 = 5+3$. Ama cümlelerde sıra önemli olabilir.

Bu sorunu çözmek için position embedding'ler önerildi; bu, bu makalenin önemli sonuçlarından biridir.

Soru (hocadan): Position embedding'leri daha önce nerede gördük? **Cevap:** Transformer'larda ve BERT'te. Onlar da girdiyi tek seferde aldıkları için sıra bilgisi kayboluyordu; bunu önlemek için temsiller position embedding'lerle toplanıyordu.

Slayttaki formül (ekrandan, 01:36:40):

$$m_i = \sum_j l_j \cdot A x_{ij}, \qquad l_{kj} = (1 - j/J) - (k/d)(1 - 2j/J)$$

Buradaki "$\cdot$" operatörü eleman bazlı çarpmadır. j kelimenin cümledeki sırası, J en büyük cümle uzunluğu, k embedding boyutu içindeki indis, d embedding boyutudur. Cümlenin birinci kelimesi L matrisinin birinci sütunuyla, ikinci kelimesi ikinci sütunuyla, yirmi beşinci kelimesi yirmi beşinci sütunuyla çarpılır.

Slaytta $d = 128$ ve $J = 25$ için L matrisi bir renk haritasıyla gösterilmiştir: "L'nin her bir sütunu farklı ve bir indeksi ifade ediyor. Bu sayede aynı kelimenin cümleye etkisi farklı indislerde farklı oluyor." L bir kez hesaplanır; öğrenilen ya da eğitilen bir şey değildir. Tek amacı kelimelerin cümle içindeki sıra bilgisini işe katmaktır.

Haritada yakın konumların sütunları birbirine yakındır (1 ile 2, 12 ile 13, 24 ile 25), birinci ve yirmi beşinci sütun ise birbirinden çok uzaktır. Bu, formülle doğrulanabilir (python ile hesaplandı): $d = 128$, $J = 25$ için ardışık iki sütun arasındaki Öklid uzaklığı her yerde yaklaşık 0,26'dır; 1. ve 25. sütunlar arasındaki uzaklık yaklaşık 6,27'dir. Matrisin değerleri 0,0078 ile 1 arasındadır.

> **Ders sonrası not:** Hoca, BERT'te bu yapının sinüs ve kosinüslerle elde edildiğini söyledi. Sinüs-kosinüs konum kodlaması özgün Transformer makalesindedir; BERT ise büyük olasılıkla öğrenilen (learned) position embedding'ler kullanır. Temel mantık (yakın konumlar benzer, uzak konumlar farklı temsil) aynıdır.

3.5 Sonuçlar

Makalede 20 görev üzerinde oldukça iyi başarı elde edilmiştir. Tablolarda her bir hop'ta (üç hop çalıştırılmış) hangi cümlenin aktivasyonunun yüksek olduğu mavi renkle gösterilir. Bazı örneklerde hop'lar arasında farklı hafıza bloklarına geçilir; bazılarında hep aynı cümlede kalınır. Cevap için tek bir cümle yetiyorsa aynı yerde kalınır; birden fazla cümle gerekiyorsa sistem birden fazla yere erişebilir. Destekleyici cümleler verilmese de sistemin bu cümleleri bulabildiği gösterilmiştir.

4. Arka Kapılar Üzerine Tekrar

Bu "harika" sonuçlara önceki derste konuşulduğu gibi itiraz geldi. Hoca konuyu genel bir çerçeveye oturttu.

Gözetimli öğrenmede algoritmaya **ne** yapması gerektiğini söyleriz: "Şöyle bir girdi gelirse şöyle bir çıktı üret." Ama **nasıl** yapması gerektiğini söyleyemeyiz. Bu kötü bir şey değildir: nasıl yapılacağını söyleyebilseydik zaten onu doğrudan koda dökerdik ve öğrenmeye gerek kalmazdı. İki sayıyı toplamak için makine öğrenmesi algoritması geliştirilmez, çünkü nasıl toplanacağı bilinir. Makine öğrenmesi, bir kişinin kim olduğunu tanımak ya da ses sinyalini metne çevirmek gibi nasıl yapılacağını kodlayamadığımız durumlar içindir.

Sıkıntılı taraf şudur: algoritma problemi hiç aklımıza gelmeyen bir yolla, arka kapıdan çözebilir. Önceki derste konuşulan çalışmalarda, test verisinden bazı bilgiler çıkarıldığında bile sistemler çok iyi sonuç vermeye devam etti. Demek ki istenen akıl yürütmeyi (reasoning) yapmıyorlar; farkına varmadığımız küçük ipuçlarını kullanıyorlar. Entailment örneği: "Parkta oynayan kediler var" ile "Parkta oynayan hayvanlar var" gibi daha genel ifade içeren bir cümle çifti varsa sistem bunu doğrudan "entailment" sınıfına atabilir.

Bu problemi çözmek kolay değildir. Önemli olan, algoritmayı "çalışıyor" diye kara kutu olarak bırakmamak, nasıl çalıştığına dair ipuçları toplamak ve arka kapı kullanıp kullanmadığını görmektir. Diğer bir husus veri kümesidir: veri kümesi buna izin veriyorsa algoritma kolayca bir arka kapı bulur. Bu yüzden veri kümesi oluşturulurken de, oluşturulduktan sonra da arka kapı olup olmadığı test edilmelidir. Sık kullanılan benchmark veri kümelerinin aslında iyi birer benchmark olmadığını gösteren yayınlar ve veri kümelerindeki arka kapıları bulmak için geliştirilen sistemler ortaya çıkmaya başladı.

5. QANet ve SQuAD

5.1 Problem

bAbI'de problem, sorunun cevabının hangi kelime olduğunu bulmaktı. SQuAD veri kümesinde problem farklıdır: bir paragraf ve paragrafa yönelik bir soru vardır. İstenen, cevabın paragrafın kaçıncı kelimesinde başlayıp kaçıncı kelimesinde bittiğidir. Yani paragraftaki n kelimenin her biri için hem cevabın başlangıcı olma olasılığı hem de cevabın sonu olma olasılığı bulunur; cevap, seçilen başlangıç ile son arasındaki kısımdır. Hoca bunun oldukça büyük bir veri kümesi olduğunu söyledi.

> **Ders sonrası not:** Hoca, yanılmıyorsa SQuAD'ın haber ve Wikipedia paragraflarından oluşturulduğunu söyledi. SQuAD paragrafları büyük olasılıkla yalnız Wikipedia makalelerinden alınmıştır.

5.2 Mimari

Slayttaki tanımlar ve şekil (ekrandan, 02:00:00; QANet, ICLR 2018):

- Paragraf (context, C) n, soru (question, Q) m kelimeden oluşur.
- Her kelime, kelime bazlı (GloVe) 300 ve karakter bazlı 200 boyutlu, toplam 500 boyutlu bir embedding ile temsil edilir. Bu hem paragrafın hem sorunun kelimeleri için yapılır.
- Stacked Embedding Encoder blokları her kelimeyi $d = 128$ boyutlu bir kodlamaya çevirir; hoca bunun BERT'teki yapıya benzediğini söyledi.
- Ortada Context-Query Attention katmanı vardır. Çıktısı $4d \times n$ boyutlu $[C; A; C \odot A; C \odot B]$'dir.

- Ardından üç Stacked Model Encoder bloğu gelir; çıkışları M_0, M_1, M_2 olarak adlandırılır. Bunlardan Concat → Linear → Softmax ile başlangıç olasılığı (start probability), aynı yapıyla bitiş olasılığı (end probability) hesaplanır.

Hoca karakter bazlı kısmı one-hot karakter kodlaması olarak anlattı. Karakter sözlüğünün 200 farklı karakter içerdiğini varsayarsak, bir kelime 200 boyutlu bir vektörle gösterilir ve sıfırdan büyük elemanlar o karakterin kelimede kaç kez geçtiğini gösterir. Böylece GloVe anlamsal yapıyı, karakter kısmı sözdizimsel (sentaktik) benzerlikleri tutar.

> **Ders sonrası not:** QANet makalesinde karakter bazlı 200 boyutlu temsil büyük olasılıkla bir sayım vektörü değildir: her karakter eğitilen 200 boyutlu bir vektörle gösterilir ve kelimenin karakter vektörleri üzerinde konvolüsyon ve maksimum alma ile tek bir 200 boyutlu vektör elde edilir. Hocanın anlatımı sezgi olarak doğrudur (karakter bilgisi sentaktik benzerliği taşır), ama yapı farklıdır.

5.3 Context-query attention

Buradaki attention bir self-attention değildir. Self-attention, bir şeyin kendi parçalarının birbirleriyle ilişkisidir. Context-query attention ise paragraftaki n kelimenin sorudaki m kelimeyle ilişkisini, yani en çok hangi kelimelerin hangileriyle ilişkili olduğunu ortaya çıkarır.

Slayttaki tanımlar (ekrandan, 02:00:00):

- Paragrafın i. kelimesinin (c) sorunun j. kelimesine (q) benzerliği: $S_{ij} = W_0[c; q; c \odot q]$.
- W_0: $3d$ boyutlu öğrenilen bir vektör.
- S: $n \times m$ boyutlu bir matris. Satırları paragrafın kelimelerine, sütunları sorunun kelimelerine karşılık gelir.
- $\bar{S}$: S'nin softmax ile satır normalizasyonu. $\bar{\bar{S}}$: softmax ile sütun normalizasyonu.
- $A = \bar{S} \cdot Q^\top$
- $B = \bar{S} \cdot \bar{\bar{S}}^\top \cdot C^\top$
- Q: $d \times m$, C: $d \times n$.
- Slayttaki soru: "A, B: neyi ifade eder?"

Hoca normalizasyonların anlamını açıkladı. Satır normalizasyonunda her satır, paragrafın bir kelimesinin sorunun her kelimesine benzerliğini gösterir; normalizasyon en büyük değerleri belirginleştirir. Sütun normalizasyonunda her sütun, sorunun bir kelimesinin paragrafın her kelimesine benzerliğini gösterir. A'nın içinde $\bar{S}$ (benzerlikler) ve Q (soru) vardır; çarpımları paragrafın soruya en çok benzeyen kısımlarını öne çıkarır. Böylece ortaya çıkan temsilin içinde hem paragraf hem soru bulunur. Soru Q, mimarinin üst kısmında doğrudan görünmez; işe bu attention katmanında girer.

> **Ders sonrası not:** Hoca $4d \times n$ ifadesindeki d için "şuradaki 500" dedi. Slayta göre attention katmanına giren her kelime $d = 128$ boyutludur, dolayısıyla çıktı $4 \cdot 128 \times n = 512 \times n$ boyutludur. Ayrıca hoca W_0'dan "matris" diye söz etti; slaytta W_0 $3d$ boyutlu bir vektördür.

Hoca bu yapıların "ad hoc" olduğunu vurguladı: mantıklı ama gelişigüzel kurulmuşlardır ve bu derin öğrenme mimarilerinin büyük bir problemidir. Yazarlar yapıyı başka türlü kurduklarında o veri kümesinde bu kadar başarılı olamamışlardır. Ama başka bir veri kümesinde de bu yapının diğerlerinden daha iyi olacağını söylemek güçtür. Kabaca kritik olan, girdi ile çıktı arasındaki bu attention mekanizmasıdır. Bu mekanizma daha sonra BERT mimarilerinde self-attention'a evrildi; daha doğrusu self-attention da işin içine katıldı.

Son olarak p_1 her kelimenin başlangıç kelimesi olma, p_2 bitiş kelimesi olma olasılığıdır. Burada yine çeşitli öğrenilen matrisler vardır. Buradaki M_0 ve M_1 memory networks'teki hikaye satırları değil, model encoder bloklarının çıkış vektörleridir.

6. Bu Çalışmalar Neden Önemli?

Hoca genel resmi şöyle çizdi. Memory networks iyi bir amaç için oluşturuldu, sonra içinde problemli yerler olduğu görüldü. Önce memory networks, sonra end-to-end memory networks geldi. Bunlar şöyle çalışmalardır: kendileri sisteme çok iyi bir çözüm sunmaz, ama içlerindeki

bazı parçalar ileriki araştırmacılara esin kaynağı olur. Position embedding'ler ve attention bunun örneğidir; kendileri çok büyük başarı elde edemeseler bile sonraki çalışmaları etkilemişlerdir.

> **Hocanın vurgusu:** Bu çalışmalar "reasoning'i çözüyorlar" ya da "bilgisayarlar artık bir paragrafı anlayabiliyor" gibi sonuçlar için gerçekçi değildir. Bu tür sonuçlara gitmek yerine mimarilerin güzel yanlarını alıp sonraki çalışmalarda kullanmak gerekir. Bu yapıların derste ayrıntılı anlatılmasının sebebi budur.

7. Sohbet ve Sanal Asistan Uygulamaları

Önceki derste şiir üretimi konuşulmuştu. Bu derste bir uygulama alanı olarak sohbet ve sanal asistan sistemlerine geçildi. İkisi farklı şeylerdir.

Sohbet sistemi, bir sistemle sohbet edebilmek içindir. Bir amacı yoktur ya da amacı hoşça vakit geçirmektir. İyiliği ya da kötülüğü kolayca, sayısal olarak ölçülemez. Hocaya göre bunlar gerçekleştirmesi çok zor sistemlerdir; birisiyle gerçekten sohbet edebilen bir sistem geliştirmek "korkunç zor" bir şeydir.

Sanal asistanlar ise hedeflidir: bir ürün ya da hizmet bulmak, satın almak, bir şey aramak, yardım almak, bir problemi çözmek gibi. Hocaya göre asistanlar pratikte daha önemlidir. Hedefin olması, kullanıcının söyleyebileceği şeylerin uzayını kısıtlar. Sohbet botuyla konuşurken onu kandırmaya ya da kızdırmaya çalışmak gibi çok geniş bir uzayda hareket edilebilir. Ama kendi hesabınızdan başkasına havale yaparken sistemin yanlış çalışmasını hiç istemezsiniz; onu kandırmaya çalışmaz, söylediklerinizi düzgün söylersiniz. Bu durum işin önemine göre değişse de uzayın kısıtlandığı bir gerçektir. Bu yüzden görev tabanlı sistemler sohbet amaçlılardan çok daha kolaydır.

7.1 ELIZA

İlk sohbet sistemi, belki ilk chatbot, ELIZA'dır. Slayta göre (ekrandan, 02:13:05) 1965'te Joseph Weizenbaum tarafından geliştirilmiştir ve bir terapisti simüle eder. Slayttaki diyalogda insanın cümleleri kırmızı, ELIZA'nınkiler siyah büyük harflerle yazılmıştır:

```
Men are all alike.
IN WHAT WAY?
They're always bugging us about something or other.
CAN YOU THINK OF A SPECIFIC EXAMPLE?
Well, my boyfriend made me come here.
YOUR BOYFRIEND MADE YOU COME HERE?
He says I'm depressed much of the time.
I AM SORRY TO HEAR THAT YOU ARE DEPRESSED.
It's true that I am unhappy.
DO YOU THINK COMING HERE WILL HELP YOU NOT TO BE UNHAPPY?
I need some help, that much seems certain.
TELL ME MORE ABOUT YOUR FAMILY.
My mother takes care of me.
WHO ELSE IN YOUR FAMILY TAKES CARE OF YOU?
My father.
YOUR FATHER
```

Hoca hangisinin insan olduğunu sordu; doğru cevap kırmızı olanlardır.

ELIZA'nın çalışma mekanizmasına bakıldığında tüm etkileyiciliğini biraz kaybeder (ekrandan, 02:14:00). Uyan bir şablon varsa dönüşüm uygulanıp cevap verilir:

```
(X me Y)                  ---> (X you Y)
(I remember X)            ---> (Why do you remember X just now?)
(My {family-member} is Y) ---> (Who else in your family is Y?)
(X {family-member} Y)     ---> (Tell me more about your family)
```

Bunlar örnek şablonlardır; gerçekte daha fazlası vardır. Hiçbir şablona uymayan bir cümlede genel amaçlı cevaplardan biri rastgele seçilir. Bu, tamamen refleks mimarisinde tanımlanmış

ama esnek şablonlu bir yapıdır. Kişinin söylediğini ona geri söylemek bir psikoterapi tekniğidir: kişi aynı şeyi başkasının ağzından duyduğunda olayı farklı algılayabilir. ELIZA'nın geçmişi yoktur; yalnız anlık cümleye bakar.

7.2 PARRY

1971'de Colby'nin geliştirdiği PARRY paranoyak bir kişiyi taklit eder. ELIZA'ya benzer ama varsayılan bir kişiliği ve ruh hali vardır ve bunlar kullanıcının söylediklerine göre değişir. Yani sistemin bir durumu (state) vardır. ELIZA'dan farkı geçmişin hesaba katılmasıdır: aynı şeyi farklı zamanlarda söylediğinizde etkisi farklı olabilir. Oluşturmak için çok uğraşılmıştır: ne duyduğunda bunu nasıl algılayacağı, cevaplarının buna göre nasıl değişeceği tanımlanmıştır. Yine şablon ve kural tabanlı bir yapıdır, ama içinde çok sayıda değişken vardır.

8. Diyalog Derlemleriyle Cevap Bulma ve Soft Cosine

8.1 En benzer ifadenin cevabı

Kuralları oluşturmak ve yönetmek pahalıdır. Daha basit bir yol, diyalog derlemlerini kullanmaktır (ekrandan, 02:20:00): Twitter diyalogları, WhatsApp diyalogları, film ve dizi diyalogları. WhatsApp mesajlarını indirmek muhtemelen mümkün değil, ama Twitter'dan veri indirilebilir ve film-dizi altyazıları kullanılabilir: birisi bir şey söylüyor, karşısındaki başka bir şey söylüyor.

Derlem satır satır ilerleyen bir diyalog gibidir; her satır bir öncekine cevap olarak yazılmıştır (A, B, A, B, ...). Kullanıcının söylediğine (q) derlemde (D) en benzeyen ifade (t') bulunur:

$$t' = \arg\max_{t \in D} \frac{q^\top t}{\|q\|\|t\|}$$

Bu, q ile t arasındaki kosinüs benzerliğidir. Cevap olarak t''ye verilen ifade döndürülür. t' için birden fazla cevap varsa biri rastgele seçilebilir. Derlem olarak diyaloglar yerine kitap, ansiklopedi gibi kaynaklar da kullanılabilir.

Aynı yapı sıkça sorulan sorular (SSS) için de kullanılabilir. Elinizde örneğin bin tane soru-cevap çifti olsun; insanlar listeyi okumadan doğrudan soru soruyor. Kullanıcı bir soru sorduğunda ona en benzeyen soru bulunur. Benzerlik bir eşiğin üstündeyse onun cevabı doğrudan verilir. En benzeyen bile yeterince benzemiyorsa, cevabın SSS'de olmadığı düşünülüp soru gerçek bir insana yönlendirilebilir.

8.2 Soft cosine

q ve t one-hot (kelime torbası) gösterimiyle çalıştığı için anlamsal yapıyı hiç dikkate almaz. Hocanın daha önceki derslerden örneği: "Cimbom kazandı" ile "Aslan yendi" makine açısından tamamen farklı iki cümledir, ama anlamca birbirine yakındır; ikisi de spor manşetinde geçebilir.

Bunu yakalamak için kosinüs "yumuşatılabilir". Slayttaki formüller (ekrandan, 02:21:05):

$$\cos(X, Y) = \frac{X^\top Y}{\sqrt{X^\top X}\sqrt{Y^\top Y}}, \qquad X^\top Y = \sum_{i=1}^{n} x_i y_i$$

$$\cos_M(X, Y) = \frac{X^\top M Y}{\sqrt{X^\top M X}\sqrt{Y^\top M Y}}, \qquad X^\top M Y = \sum_{i=1}^{n}\sum_{j=1}^{n} x_i m_{ij} y_j$$

Soft cosine'da çarpımın arasına, hem payda hem paydada, bir M matrisi girer. M derlemdeki her kelimenin her kelimeye benzerliğini tutar. Bu, QANet'teki paragraf kelimesi-soru kelimesi benzerlik matrisine benzer, ama burada sözlükteki bütün kelime çiftleri içindir. Slaytta M'yi hesaplamak için iki yöntem verilmiştir:

$$m_{ij} = \max(0, \cos(v_i, v_j))^2 \quad (v : \text{kelimelerin word2vec vektörü})$$

$$m_{ij} = \alpha \cdot \left(1 - \frac{\text{Levenshtein}(w_i, w_j)}{\max(\|w_i\|, \|w_j\|)}\right)^{\beta} \quad (\text{kelimeler arası edit distance})$$

Bu ikisi birleştirilerek de kullanılabilir, örneğin edit distance ile word2vec benzerliği ağırlıklandırılarak. Hoca fastText'te de bu tür bir yapı olduğunu söyledi. Kaynak: aclweb.org/anthology/S17-2003.

M'nin sorunu boyutudur: sözlük × sözlük. Bunu saklamak ve hesaplamak sıkıntılı olabilir. Ama en önemli 5000 kelime seçilirse 5000×5000'lik bir matris yönetilebilir. Basit yöntemlerle yapılan uygulamalarda soft cosine'ın kosinüse göre fayda sağladığı görülmüştür.

9. Diyalog Üretimi ve Microsoft Tay

Aynı diyalog derlemiyle cevap seçmek yerine cevap ürettirilebilir. Bu klasik bir sekanstan sekansa (sequence to sequence) problemidir; makine çevirisinde olduğu gibi bir cümle verilir, sistem bir cümle üretir. Klasik RNN mimarileri ya da daha gelişmiş hâlleriyle yapılabilir. Slayttaki yapıda (RNN'lerle diyalog öğrenme) kullanıcının bütün söyledikleri tek bir vektöre indirgenmiştir. Bu vektör hem ilk kelime hem sonraki kelimeler tahmin edilirken sisteme verilir. Eğitim için "şunu verdiğinde şunu söylesin" biçiminde bir veri kümesi kurulur.

Microsoft'un Tay adlı sohbet robotunun iyi niyetli tarafı, kullanıcılarla yaptığı diyaloglardan da öğrenmesiydi: sistem çalışmaya başladıktan sonra da eğitim sürüyor, kullanıcılarla konuşmalar yeni bir diyalog derlemine dönüşüp eğitim kümesine katılıyordu. Ama sistem kısa süre sonra kapatıldı.

Soru (hocadan): Tay neden kapatılmış olabilir? **Cevap (bir öğrenciden):** Irkçı söylemlerde bulunuyordu; insanlar onu farklı yönlere yönlendirdi. Hoca onayladı: sistemin konuşulanlardan öğrendiği duyulunca insanlar ona çok farklı şeyler öğretmiş, eğitim kümesi kötü öğrenilmiş bir derleme dönüşmüş.

Kullanıcı etkileşimlerinden öğrenmenin iyi tarafları kesinlikle vardır, ama kötü niyetli kullanıcılar kritik bir konudur. Sistemleri kandırmaya, yanlış yönlendirmeye çalışan kullanıcılar her zaman olacaktır. Makineler karar vermeye başladıkça bu girişimler hiç durmayacaktır.

10. Görev Tabanlı Sanal Asistanlar

10.1 Genel yapı

Buraya kadarki sistemler kabaca genel amaçlıydı. Bundan sonrası görev tabanlı sistemlerdir.
Slayttaki maddeler (ekrandan, 02:27:10; kaynak: web.stanford.edu/~jurafsky/slp3/slides/convagents1.pdf)

- Goal oriented, task based, frame based.
- Genel yapı 1977'den beri değişmedi: GUS.
- Bir uygulama (takvim yönetimi, bilet bulma, öneride bulunma vb.) için yardımcı sistemler.
- Yapılabilecek işlemlerin türleri belirlenir.
- Her işlem türü için gerekli bilgiler (slot) ve türleri (zaman, sayı, şehir vb.) belirlenir.
- Bu bilgileri almak için sorular belirlenir.
- Bir sıra ile bilgiler alınır.

Süreçte bir sorun çıkarsa belli yönlerde geri dönülür. Örneğin pizza siparişi veren biri için sürecin tamamlanması adına hangi bilgilerin alınacağı ve türleri belirlenir.

Uçak bileti uygulamasında slot, tür ve soru üçlüleri tanımlanır: kalkış şehri (origin city, türü city) için hangi soru sorulacak, varış şehri için hangisi; kalkış saati, kalkış tarihi, havayolu vb. Gerçek bir uygulamaya kredi kartı, kullanıcının adı, yaşı, cinsiyeti gibi daha birçok slot eklenebilir.

Bu kadar sıkı bir çerçevenin sebebi ticari uygulamalardır: kullanıcının sistemle neler yapabileceği çok net olmalı, hiçbir şey muğlak bırakılmamalıdır.

10.2 Makine öğrenmesi ve NLP nerede?

- **Kullanıcının niyeti (intent):** Kullanıcının yapabileceği birçok işlem var; hangisini yapmak istiyor?
- **Slotları ve değerlerini belirlemek:** Kullanıcının ifadesinden slot değerlerini çıkarmak.

Örnek: "Salı sabah Ankara'ya giden uçuşları gösterebilir misin?" Niyet: uçuş sorgulama. Kalkış şehri: kullanıcının şehri. Kullanıcının nerede olduğu biliniyorsa (asistan cep telefonunda çalışıyor olabilir) İstanbul varsayılabilir ya da teyit için sorulabilir. Kalkış tarihi: bugünden sonraki salı. Kalkış zamanı: sabah, yani aşağı yukarı belli saatler arası. Varış şehri: Ankara. Bütün slotlar doluysa veritabanına sorgu gönderilip cevap cümleye çevrilir.

Kullanıcı yalnız "Ankara'ya giden uçuşlar hangileri?" derse (ekrandan, 02:30:00) niyet aynıdır ama bazı slotlar boştur:

```
Ankara'ya giden uçuşlar hangileri?
  - Niyet: Uçuş sorgulama
  - Kalkış şehri: Kullanıcının şehri -> İstanbul
  - Kalkış tarihi: ??
  - Kalkış zamanı: ??
  - Varış şehri: Ankara
Kalan boşlukları doldurmak için soru sor:
  - hangi tarih için uçuş arıyorsunuz?
  - İstanbul'dan Ankara'ya hangi tarih için uçuş arıyorsunuz?
```

İkinci soru daha iyidir: hem eksik bilgiyi sorar hem de sistemin çıkardığı (kullanıcının söylemediği) kalkış şehrini teyit eder. Kullanıcı itiraz etmezse İstanbul kullanılabilir. Kullanıcı "Hayır, İzmir'den Ankara'ya gitmek istiyorum" derse bu düzeltmenin de anlaşılması gerekir. Makine öğrenmesi ve NLP bu tür yerlerde kullanılır.

10.3 Kullanıcıya anlaşıldığını hissettirmek

Slayttaki örnek (ekrandan, 02:31:20; kaynak: web.stanford.edu/~jurafsky/slp3/25.pdf):

```
(K) 3 büyük boy karışık pizza istiyorum.
(S) 3 küçük karışık pizza mı istiyorsunuz?
(S) 3 büyük boy karışık pizza siparişiniz alınmıştır. Başka isteğiniz?
(S) 3 küçük boy karışık pizza siparişiniz alınmıştır. Başka isteğiniz?
(S) Başka isteğiniz?
```

Sistem doğrudan "Başka isteğiniz?" derse kullanıcı ne olduğunu bilemez: istediğim anlaşıldı mı, sipariş gönderilecek mi, para çekildi mi? "3 büyük boy karışık pizza siparişiniz alınmıştır. Başka isteğiniz?" gibi, sistemin anladığını karşı tarafa gösteren cevaplar daha mantıklıdır. Hiçbir şey hissettirmeden doğrudan bir sonraki duruma (ödeme, başka istek alma) atlamak yerine önce teyit edip sonra süreci sürdürmek daha makuldür.

Anlaşılmamak kısmen ya da tamamen olabilir. Kısmen anlaşılan bir ifadede anlaşılanı söyleyip eksik olanı sormak gerekir (örneğin "Salı günü nereye gitmek istiyorsunuz?"). Tahmin yapıldıysa ("Ankara'ya" gibi) bu, düzeltilebilir biçimde sorulmalıdır. Kullanıcının yanlış anlaşılan bir şeyi düzeltmek istediğini fark etmek de oldukça kritiktir.

10.4 Araçlar ve neden sanal asistan?

Bu tür sanal asistanları flow biçiminde kolayca oluşturmayı sağlayan ticari araçlar vardır; örneğin bir e-ticaret sitesi için. Hocanın andıkları: Dialogflow (yanılmıyorsa Google'ın satın aldığı bir firma), Amazon'un aracı ve IBM Watson eklentileri. Diyalog veri kümeleri de bulunabilir ya da internetten taranarak (crawl) toplanabilir.

Pizza ya da uçak bileti için yazmak varken neden sanal asistan kullanılsın?

- **Sesli kontrol:** Evdeyken ya da arabadayken sesle komut verilebilir. Konuşmadan metne (speech to text) teknolojileri iyi olduğu için klavyeye iyi bir alternatiftir.
- **Çok seçenek:** Basit bir arayüzde 255 çeşitten birini seçmek gerektiğinde kullanıcının hepsini okuması zahmetlidir. Kullanıcı ne istediğini söyler, sistem olası seçenekler içinde hangisine en yakın olduğunu bulur. Seçenekleri ifade etmenin çok sayıda yolu vardır.

Hocanın değerlendirmesi: çoğu görevde insan performansının çok gerisindeyiz. Türkçe için veri kümesi ve uygulama talebi var, ama piyasada büyük bir boşluk var. Mevcut uygulamalar çok sıkı tanımlı kısımlarda başarılıdır ("Dolar bugün kaç lira?" sorusuna çok iyi cevap verirler), daha karmaşık isteklerde doğal olarak sorunlu çalışırlar. Bu normaldir: ticari uygulamalarda yanlış işlem çok büyük sıkıntıdır, doğru çalışma oranının çok yüksek olması gerekir. Bu yüzden tasarımcılar ve iş tarafı çok çekingen davranır ve çok sayıda kural seti kurar. Bu da kullanılabilirliği biraz zorlaştırabilir.

> **Hocanın vurgusu:** Bu bölümde çok teknik bir şey yoktur. Kritik olan, görev tabanlı sistemlerin slot-soru yapısının hâlâ aynı olmasıdır; iş (business) tarafındaki kaygılar yüzünden daha geniş yapılar kurulamamaktadır.

Örnek Sorular

Soru 1 (derste sözlü, 00:13:07): Markov zinciriyle cümle üreten bir program deterministik midir? Aynı başlangıç kelimesiyle defalarca çalıştırıldığında farklı sonuçlar alınıyorsa bunu ne sağlar?

> **Çözüm:** Deterministik değildir. Geçiş olasılıkları derlemden bir kez hesaplanır; her adımda mevcut kelimeden sonra gelebilecek kelimeler arasından olasılıklarına göre ağırlıklı rastgele bir seçim (weighted choice) yapılır. Bu yüzden aynı başlangıçla farklı cümleler çıkabilir.

Soru 2 (derste sözlü, 00:20:44): Üç kelimeden cümle üretmek için bir T5 modeli eğitilecek. Türkçe eğitim verisi nasıl oluşturulabilir?

> **Çözüm:** Rastgele bir cümle alınır, içinden rastgele üç kelime seçilir. Bu üç kelime girdi dizisi, cümlenin kendisi çıktı dizisi olur. T5'in görevleri sekanstan sekansa olduğu için bu yapı doğrudan kullanılabilir.

Soru 3 (derste sözlü, 00:28:39): Kelime tabanlı Markov zinciri ile karakter tabanlı LSTM'i karşılaştırıp "Markov daha iyi" sonucuna varmak neden tam adil değildir?

> **Çözüm:** İki şey birden değişmektedir: temsil birimi (kelime/karakter) ve yöntem (Markov/LSTM). Karakter LSTM, karakterlerden doğru kelime üretmeyi az veri ve kısa eğitimle öğrenememiş olabilir. Adil bir karşılaştırma için LSTM de kelime tabanlı kullanılmalıdır.

Soru 4 (ekrandan, 01:10:00): bAbI örneği: "1 Mary banyoya gitti. 2 Sandra bahçeye gitti. 3 Daniel bahçeye gitti. 4 Daniel ofise gitti. 5 Sandra oradaki sütü aldı. 6 Sandra sütü oraya bıraktı. 7 Süt nerede?" Cevap ve cevapla ilgili cümleler nelerdir?

> **Çözüm:** Cevap "bahçe". Slayttaki indisler 5, 6, 2: Sandra sütü aldı (5) ve oraya bıraktı (6); "orası" Sandra'nın gittiği yer, yani bahçedir (2).

Soru 5 (derste sözlü, 01:14:45): $o_1 = \arg\max_{i=1,\ldots,N} s_O(x, m_i)$ ifadesindeki argmax ne demektir, o_1 neyi bulur?

> **Çözüm:** argmax, fonksiyonun değerini en büyük yapan argümanı döndürür (argmin en küçük yapanı). Burada i 1'den N'ye (hafızadaki cümle sayısı) kadar denenir ve soruya (x) en çok benzeyen hafıza satırının indisi o_1 olarak seçilir.

Soru 6 (ekrandan, 01:20:00; derste sözlü, 01:13:13): $s(x, y) = \Phi_x(x)^\top U^\top U \Phi_y(y)$ ifadesinde Φ çıktısı $d \times 1$, U ise $n \times d$ boyutludur. s'in çıktısının boyutu nedir ve neyi ifade eder?

> **Çözüm:** $(1 \times d)(d \times n)(n \times d)(d \times 1) = 1 \times 1$. Çıktı tek bir sayıdır ve x ile y string'lerinin öğrenilen U uzayındaki benzerliğini ifade eder.

Soru 7 (derste sözlü, 01:34:18): Kelime torbası ile $m_i = \sum_j Ax_{ij}$ biçiminde hesaplanan cümle temsilinde, cümledeki kelimelerin yerleri değiştirilirse m_i değişir mi?

> **Çözüm:** Değişmez. Kelimelerin vektörleri toplanmaktadır ve toplama sırası sonucu değiştirmez ($3 + 5 = 5 + 3$). Bu yüzden End-to-End Memory Networks'te position embedding ($m_i = \sum_j l_j \cdot Ax_{ij}$) önerilmiştir.

Soru 8 (derste sözlü, 01:35:05): Position embedding'leri daha önce hangi mimarilerde gördük, neden gerekiyorlardı?

> **Çözüm:** Transformer'larda ve BERT'te. Bu modeller girdiyi tek seferde aldıkları için sıra bilgisi kaybolur; temsiller position embedding'lerle toplanarak kelimelerin konumu işe katılır.

Soru 9 (ekrandan, 02:00:00): QANet'te $A = \bar{S} \cdot Q^\top$ ve $B = \bar{S} \cdot \bar{\bar{S}}^\top \cdot C^\top$ neyi ifade eder?

> **Çözüm:** S, paragraf kelimeleri ile soru kelimeleri arasındaki benzerlik matrisidir. $\bar{S}$ satır normalizasyonuyla her paragraf kelimesi için soru kelimelerine dağılmış ağırlıklar verir; A, her paragraf kelimesi için sorunun bu ağırlıklarla toplanmış temsilidir (paragraftan soruya attention). B, sütun normalizasyonu da kullanılarak her paragraf kelimesi için paragrafın soruyla ilgili kısımlarından toplanmış bir temsil verir. Hoca, A'nın içinde benzerlikler ve soru bulunduğunu, çarpımın paragrafın soruya en çok benzeyen kısımlarını öne çıkardığını anlattı; B'nin ayrıntısı derste açılmadı.

Soru 10 (ekrandan, 02:13:05): ELIZA diyaloğunda ("Men are all alike." / "IN WHAT WAY?" ...) hangi satırlar insana aittir?

> **Çözüm:** Kırmızı (küçük harfli) satırlar insanın, siyah büyük harfli satırlar ELIZA'nındır. Derste öğrenciler doğru tahmin etti.

Soru 11 (derste sözlü, 02:24:20): Microsoft'un Tay adlı sohbet robotu açıldıktan kısa süre sonra neden kapatıldı?

> **Çözüm:** Tay kullanıcılarla yaptığı konuşmalardan öğrenmeye devam ediyordu. Kötü niyetli kullanıcılar ona ırkçı ve kötü söylemler öğretti; eğitim kümesi kötü bir derleme dönüştü.

Soru 12 (ekrandan, 02:31:20): Kullanıcı "3 büyük boy karışık pizza istiyorum" dedi. Sistemin verebileceği cevaplardan hangisi kullanıcıya anlaşıldığını en iyi hissettirir: "3 küçük karışık pizza mı istiyorsunuz?", "3 büyük boy karışık pizza siparişiniz alınmıştır. Başka isteğiniz?", "3 küçük boy karışık pizza siparişiniz alınmıştır. Başka isteğiniz?", "Başka isteğiniz?"

> **Çözüm:** "3 büyük boy karışık pizza siparişiniz alınmıştır. Başka isteğiniz?" İsteği doğru tekrar edip teyit eder. Yalnız "Başka isteğiniz?" kullanıcıyı belirsizlikte bırakır; "küçük" içeren cevaplar yanlış anlamayı gösterir.

Dersin Toparlaması

Dersin ilk yarısında üç kelimeden cümle üretimi ödevinin sunumları yapıldı. Küçük derlemlerde Markov zincirleri yerel olarak anlamlı kelime grupları ürettiği için çoğu zaman az veriyle eğitilen LSTM'lerden daha iyi göründü. word2vec embedding'leri fastText'ten daha anlamlı sonuç verdi. Hoca, T5 gibi bir seq2seq modeli için eğitim verisinin cümlelerden rastgele kelime seçilerek kolayca kurulabileceğini gösterdi. İkinci yarıda önceki dersten kalan soru cevaplama mimarileri işlendi. Memory networks'te benzerlik fonksiyonu ve argmax adımlarıyla destekleyici cümleler ve cevap bulunur, ama eğitimde destekleyici cümleler işaretli olmalıdır. End-to-end memory networks softmax ağırlıklı hafıza okumasıyla bu gerekliliği kaldırır ve kelime sırasını position embedding ile işe katar. QANet, SQuAD'da cevabın başlangıç ve bitiş kelimelerini paragraf-soru attention'ı ile bulur. Hoca bu çalışmaların değerinin, position embedding ve attention gibi sonraki modellere (Transformer, BERT) taşınan parçalarında olduğunu vurguladı.

Son bölümde sohbet ve sanal asistan sistemleri anlatıldı. ELIZA ve PARRY gibi kural tabanlı erken sistemlerden, diyalog derleminde kosinüs ya da soft cosine ile en benzer ifadenin cevabını döndürmeye ve seq2seq ile diyalog üretimine geçildi. Microsoft Tay örneği kullanıcıdan öğrenmenin riskini gösterdi. Görev tabanlı asistanlarda niyet tespiti, slot doldurma, eksik slotlar için soru sorma ve kullanıcıya anlaşıldığını hissettirme ele alındı. Bir sonraki derste Bölüm 7'ye, işin görsel kısmına geçilmesi planlandı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.